WEB DEV DEPARTMENT · PLATFORM ENGINEERING

GAIADA WebDesk

A centralized, multi-tenant website backend — CMS, forms, and mail — serving every client site through one uniform, contractual, AI-operable platform. Engineering blueprint, team roadmap, and milestone reference.

DOCUMENT	CODENAME	DATE	OWNER
Engineering Blueprint v1.0	**WebDesk** (placeholder)	**2026-07-23**	**Web Dev · Platform**
STATUS	CLASSIFICATION		
● DRAFT FOR REVIEW	**Internal**		

Contents

Sections are numbered; each begins on a new page. Use the PDF bookmarks or the page footer to navigate.

§00 Executive Summary

What we are building, why, and the shape of the decision already made.

WebDesk is a **Website-Backend-as-a-Service** operated in-house. Today every client website carries its own backend: a WordPress install here, a hand-rolled Node API there, `web3forms` for a contact form somewhere else. Each is provisioned, secured, and maintained separately. WebDesk collapses that into **one central backend** that all client frontends — WordPress, Astro static sites, and Node apps — consume through a single, versioned, uniform contract.

The strategic payoff is that **frontend engineers stop building backends**. The API surface is pre-built, deterministic, and contractual; a new client site becomes a frontend composed against a generated, typed SDK. The same control surface is operable three ways — by a person clicking in the ERP, or by an AI agent acting on a human request, or automatically — because every operation is one command behind one authorization model.

The shape, in one paragraph

A rebranded, self-hosted **Payload 3** instance is the content engine. Around it sit purpose-built **NestJS services** for forms, mail, media, and provisioning. Content is modeled as a **fixed vocabulary of blocks and fields composed per client**, and every tenant's schema is compiled into a **typed SDK plus an OpenAPI contract plus a human-readable content contract** that the frontend (and Claude, when building it) targets. The platform is internet-facing and lives in its own hard **trust zone**, physically separate from the ERP that holds company data. The ERP controls it over a one-way, mutually-authenticated channel. It runs on three boxes — internal ERP, a staging site, and a live site — with **one-click, versioned, reversible promotion** between staging and live.

> DECISION STATE
>
> All architecture-level decisions in this document are **locked** (see §14 Decision Log). What remains open are the four implementation-detail items flagged in §14 Open Items. This blueprint is the spec that feeds decomposition into tiered build tickets.

§01 Goals, Non-Goals & Principles

Goals

- **One backend, many frontends.** WordPress, Astro, and Node sites all consume the same central CMS, forms, and mail.
- **Deterministic, uniform contracts.** FE devs get a typed SDK and a stable API; they never hand-write backend calls or invent shapes.
- **Tri-modal operation.** Every action is executable manually (ERP click), autonomously (AI on human request), or programmatically — via one command set.
- **Hard isolation from company data.** An internet-facing platform must never be a path into the ERP's data.
- **Portable.** Runs on Hostinger KVM8 today; relocatable to AWS/GCP with config, not code, changes.
- **Smooth staging→live.** One click promotes a project between environments; one click rolls it back.

Non-Goals (v1)

- Not a public, self-serve SaaS for external customers — it is an internal platform operated by Gaiada staff and its AI workforce.
- Not a page-builder/WYSIWYG for clients — frontends are bespoke, built by our team against the contract.
- No zero-deploy runtime schema authoring — new structured collections go through an approved, versioned deploy (see §07).
- No Kubernetes in v1 — Docker Compose per box; K8s is a documented future path, not a launch requirement.

Architectural principles

P1 · CONTRACTS FIRST

The versioned contract is the product. Schema is a compilation source; SDKs are generated, never hand-written.

P2 · TWO TRUST ZONES

Internet-facing platform and internal ERP are separate zones with a one-way, authenticated seam. Breach containment over convenience.

P3 · ONE COMMAND, ONE AUTHZ

Human, AI, and automation call the same control-plane commands under the same Cerbos/RBAC and WS4 approval gates. No backdoors.

P4 · UNIFORM VOCABULARY

Flexibility lives in composition (data); uniformity lives in the fixed block/field/envelope vocabulary (code).

P5 · PORTABLE PRIMITIVES

Postgres, Redis, and an S3 API only. No proprietary managed service in the hot path.

P6 · REVERSIBLE BY DEFAULT

Every promotion is a versioned release with a snapshot and a rollback. Irreversible actions are gated.

§02 System Overview

The whole platform on one plate: the two zones, the services in each, and every seam between them.

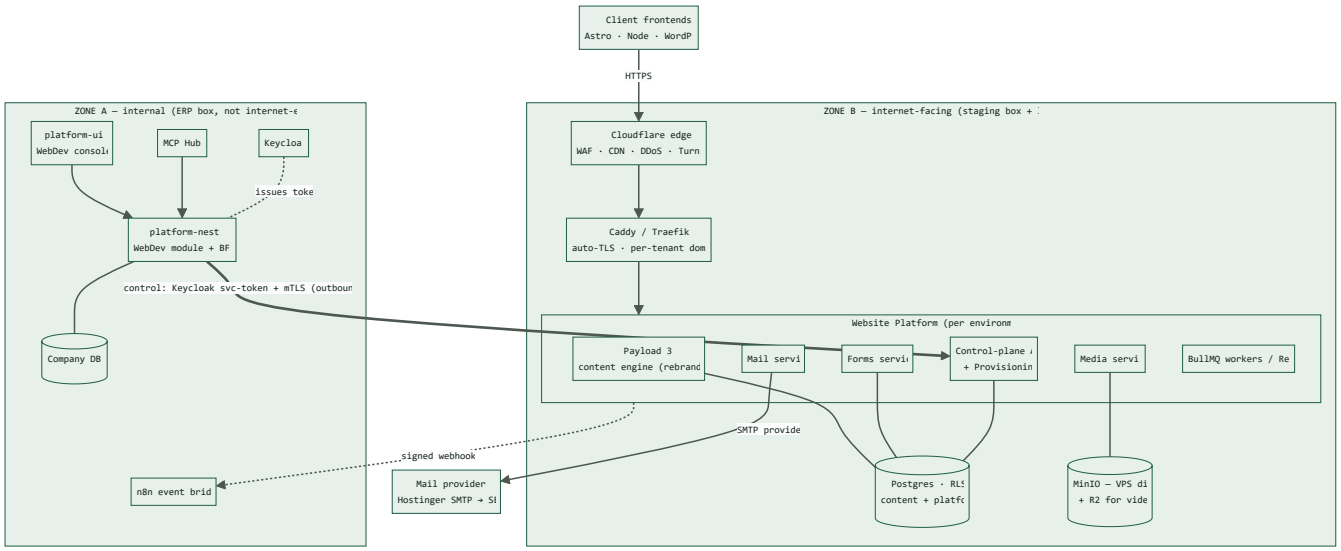

FIG. 02-1 — Master system diagram. Bold seam = control (one-way, mutually authenticated). Dotted seam = events (signed, validated). No line crosses from a client site into Zone A.

Reading the diagram

- **Client frontends** only ever reach **Zone B**, and only through Cloudflare. They authenticate with scoped, per-tenant keys.
- **Zone B** is the website platform, deployed once per environment (staging box, live box). It owns its own Postgres and object storage.
- **Zone A** is the ERP. It *controls* Zone B via outbound calls only; it never exposes company data to Zone B.
- The **control seam** carries commands (provision, deploy, promote, rotate). The **event seam** carries facts back (form received, deploy done, error) as signed webhooks into the existing n8n bridge.

§03 Trust Zones & Network

The security backbone: what is exposed, what is private, and the exact ports and directions across the boundary.

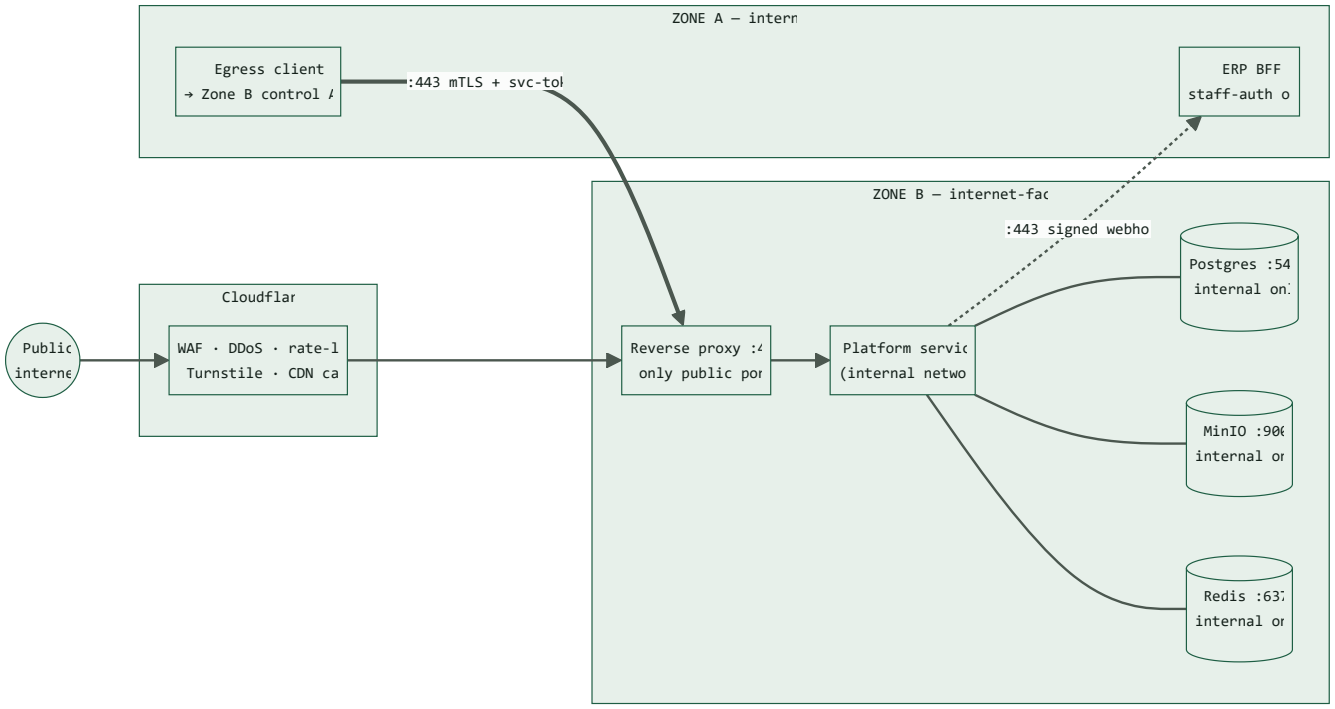

FIG. 03-1 — Network boundaries. Only `:443` is public in Zone B; datastores never leave the internal Docker network. Zone A initiates control; Zone B initiates only signed events.

Zone definitions

PROPERTY	ZONE A — ERP	ZONE B — WEBSITE PLATFORM
Exposure	Internal only (staff-auth / VPN); no public ingress	Internet-facing behind Cloudflare
Holds	Company DB, PM/HR/IT data, AI automations	Client website content, media, form submissions
Datastores	Existing company Postgres, Redis	Dedicated Postgres + MinIO + Redis (separate instances)
Trusts input from	Authenticated staff + AI via Keycloak	Untrusted public + scoped tenant keys
Blast radius if breached	Company data (highest value)	`Website + form data only`

> **CONTAINMENT INVARIANT**
>
> There is no network route and no credential by which a compromised client site, or a full compromise of Zone B, can read or write company data in Zone A. The only Zone B → Zone A path is a signed, schema-validated webhook that the ERP treats as untrusted input.

Boundary rules

- **Control is outbound from Zone A only.** The ERP calls Zone B; Zone B has no credentials for Zone A's APIs or DB.
- **Datastores are never exposed.** Postgres, MinIO, and Redis bind to the internal Docker network; the only public listener is the reverse proxy on `:443`.
- **mTLS + service token** on the control channel — a stolen token alone is insufficient without the client certificate.
- **Events are validated.** Zone B → Zone A webhooks are signed (HMAC) and schema-checked; the ERP never executes on their contents without validation.

§04 Deployment Topology

Three physical boxes, the containers on each, and how the platform stays portable off Hostinger.

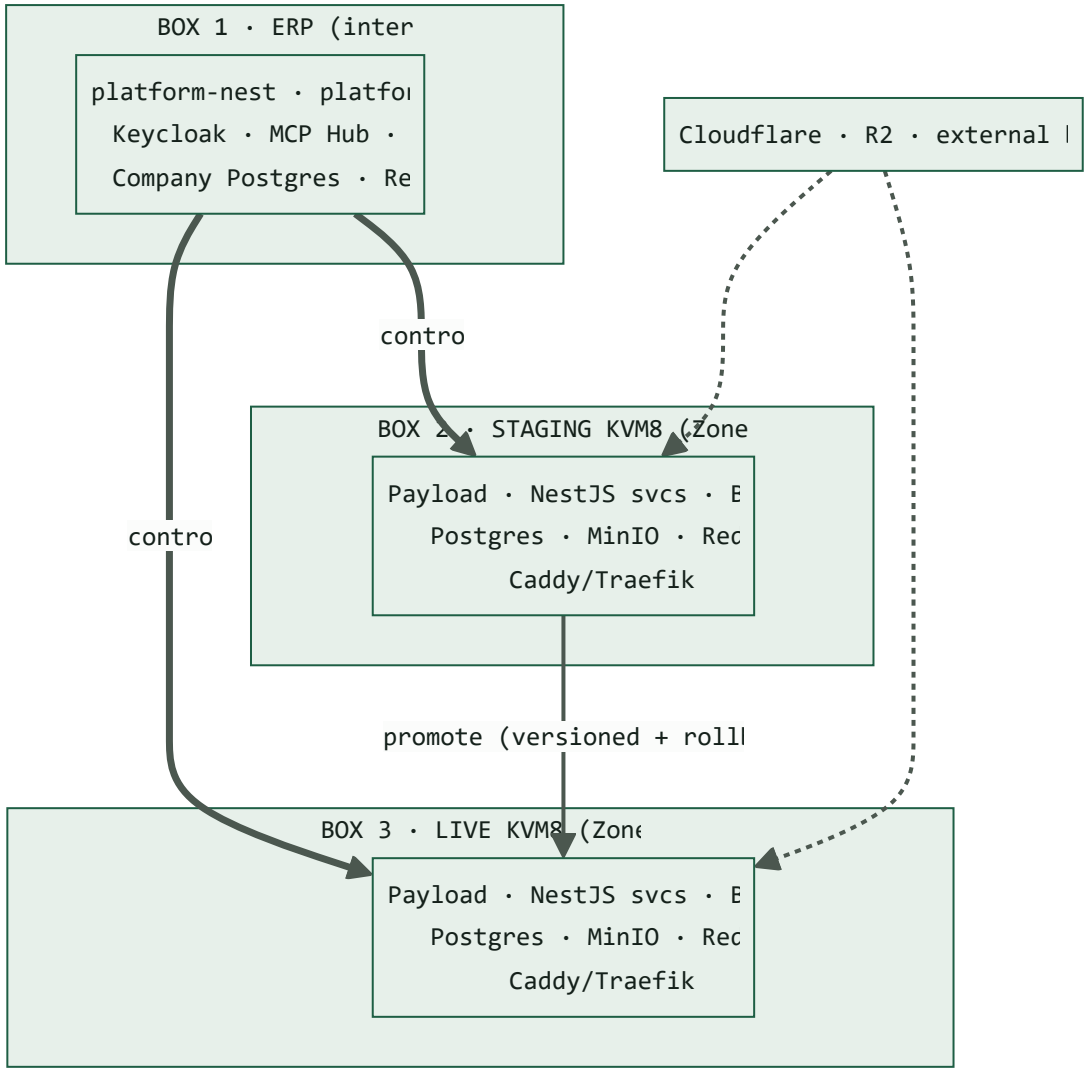

FIG. 04-1 — Three-box topology. Staging and live are identical stacks on separate boxes; the ERP controls both; promotion flows staging → live.

Per-box container inventory (Zone B)

CONTAINER	IMAGE BASIS	PURPOSE	EXPOSED
proxy	caddy / traefik	TLS termination, per-tenant domain routing	:443 public
payload	node	Content engine + admin (headless)	internal
api	node (NestJS)	Forms, mail, media, control-plane	internal
worker	node (BullMQ)	Mail send, rebuilds, media processing, promotion jobs	internal
postgres	postgres	Content + platform data (RLS)	internal
minio	minio	S3-compatible object storage on VPS disk	internal
redis	redis	Queue + cache	internal
clamav	clamav	Upload virus scanning	internal
otel	otel-collector	Telemetry export to Zone-A Grafana stack	internal

Portability

Every stateful dependency is a portable primitive: **Postgres**, **Redis**, and an **S3 API**. Moving off Hostinger is a substitution, not a rewrite:

CONCERN	HOSTINGER (V1)	AWS / GCP (FUTURE)	CHANGE REQUIRED
Object storage	MinIO on VPS disk	S3 / GCS (S3 API)	endpoint + creds
Video	Cloudflare R2	R2 / S3 / CDN	bucket config
Compute	Docker Compose	ECS / Cloud Run / K8s	compose → manifests
Mail	Hostinger SMTP	SES / Resend	provider adapter
Database	Postgres container	RDS / Cloud SQL	connection string

§05 Component Specifications

Each service specified to the same shape — purpose, technology, responsibilities, interfaces, data owned, security, and operations.

C-01 Content Engine — Payload 3

ZONE B

PURPOSE	Model, store, and serve structured content for every tenant. Rebranded, headless; its admin is not the primary surface — the ERP is.
TECHNOLOGY	Payload 3 (Next.js runtime, TypeScript), MIT-licensed, self-hosted on our Postgres. Optional telemetry disabled.
RESPONSIBILITIES	• Define collections/globals from the fixed vocabulary (§07); enforce access control per tenant. • Serve content over REST + GraphQL + Local API (used in-process by NestJS). • Manage drafts/publish states and preview tokens. • Own media records (metadata, focal point, generated sizes); binaries delegated to storage adapter (C-05).
INTERFACES	in: control-plane provisioning, editor writes · out: content read API (cached), publish webhooks
DATA OWNED	`collections`, `content`, `versions`, media records — all tenant-scoped by RLS
SECURITY	Admin never publicly exposed; access control keyed on tenant + Cerbos; secrets via env
OPERATIONS	Schema changes require boot-time recompile + migration (see §07); horizontal read scaling via CDN cache

C-02 Forms Service

ZONE B

PURPOSE	The `web3forms` replacement — accept public form submissions from any client site, validate, persist, and trigger notifications.
TECHNOLOGY	NestJS endpoint + BullMQ job on submission.
RESPONSIBILITIES	• Public `POST /v1/forms/:formId/submit` with per-tenant CORS origin allowlist. • Anti-abuse: Cloudflare Turnstile verification, honeypot field, rate limiting, size caps. • Validate against the form's declared schema (zod); sanitize; persist submission. • Enqueue notifications (mail via C-03, event to ERP) and optional autoresponder.
INTERFACES	in: public HTTPS POST · out: mail jobs, signed submission event → ERP
DATA OWNED	`form_definitions`, `submissions` (may contain PII → retention policy §10)
SECURITY	Highest-abuse surface: bot protection + rate limit + strict origin + input validation are mandatory, not optional
OPERATIONS	Submission spikes alert (§12); dead-letter queue for failed notifications

C-03 Mail / SMTP Service

ZONE B

PURPOSE	Centralized transactional mail for all tenants — form notifications, autoresponders, system mail.
TECHNOLOGY	NestJS + BullMQ; pluggable **provider adapter** (Hostinger SMTP now → SES/Resend later).
RESPONSIBILITIES	• Template rendering per tenant; from/reply-to per tenant domain. • Queue, send, retry with backoff, record delivery status. • Suppression list, bounce/complaint handling (provider-dependent).
INTERFACES	in: internal job enqueue only (never public) · out: SMTP/API to provider
DATA OWNED	`mail_templates`, `mail_log`, `suppressions`
SECURITY	No public endpoint; SPF/DKIM/DMARC per sending domain; rate limits to protect sender reputation
OPERATIONS	Provider swap = adapter config; delivery-failure alerting

C-04 Media & Object Storage

ZONE B

PURPOSE	Store and serve images, video, and file binaries for all tenants, portably and behind a CDN.
TECHNOLOGY	**MinIO** (S3 API) on the KVM8 disk for images/files; **Cloudflare R2** for video; Cloudflare CDN in front; ClamAV scan on upload.
RESPONSIBILITIES	• Accept authenticated uploads (never public write); enforce type allowlist + size caps; virus scan. • Store under per-tenant bucket/prefix with scoped credentials. • Serve public assets from a **cookieless domain** via CDN; signed URLs for private assets. • Generate/serve responsive image variants (Payload sizes or on-the-fly resizer).
INTERFACES	in: authenticated upload via Payload/NestJS · out: CDN-cached asset URLs
DATA OWNED	Object binaries (records live in Payload/Postgres)
SECURITY	Single-disk MinIO has no erasure-coding redundancy → external replication backup required (§12); cookieless serving blocks XSS pivots
OPERATIONS	Route video to R2 to protect VPS bandwidth; lifecycle rules to expire old versions

C-05 Control-Plane API & Provisioning

ZONE B

PURPOSE	The single command surface the ERP (and AI, and automation) drive. Every lifecycle action is a command here.
TECHNOLOGY	NestJS; authenticated by Keycloak client-credentials + mTLS; authorized by Cerbos; audited.
RESPONSIBILITIES	• Tenant & site lifecycle: `createTenant`, `createSite`, `setDomain`, `archiveSite`. • Schema: `proposeSchema`, `applySchema` (triggers approved deploy). • Keys: `mintKey`, `rotateKey`, `revokeKey`. • Release: `deploy`, `promote`, `rollback`, `triggerRebuild`. • Emit signed events to ERP; write immutable audit entries.
INTERFACES	in: ERP control calls (mTLS) · out: orchestrates Payload/DB/storage/CI; events → ERP
DATA OWNED	`tenants`, `sites`, `environments`, `api_keys`, `audit_log`, `releases`
SECURITY	Every command re-checks authz regardless of caller (human/AI); irreversible commands require WS4 approval token
OPERATIONS	Commands are idempotent where possible; long-running ones run as tracked jobs

C-06 ERP Integration (WebDev Module)

ZONE A

PURPOSE	The control-plane *client* and operator UI, inside the existing ERP. Holds no CMS logic.
TECHNOLOGY	platform-nest module (BFF + egress client) + platform-ui WebDev console section.
RESPONSIBILITIES	• Proxy browser requests to the control-plane API (browser never touches Zone B directly). • Render site registry, per-tenant status, submissions, storage usage, deploy/promote/rollback controls. • Hold Keycloak service credentials + client cert for the control channel. • Extend `lib/rbac.ts` with webdev capabilities; reuse company-scope + WS4 approvals.
INTERFACES	in: staff clicks, MCP tool calls · out: control-plane API (mTLS)
DATA OWNED	None authoritative — mirrors Zone B state for display; caches read models
SECURITY	Staff auth via Keycloak; the ERP is the only holder of Zone B control credentials
OPERATIONS	Degrades cleanly if Zone B unreachable (read-only status)

C-07 AI Execution Surface

ZONE A

PURPOSE	Let the AI workforce perform the same operations a human can — on human request or autonomously — with the same guardrails.
TECHNOLOGY	MCP tools on the existing Hub wrapping each control-plane command; WS4 approvals; audit log.
RESPONSIBILITIES	• Expose commands as MCP tools with typed params and least-privilege service accounts. • Draft per-tenant schemas from a client PRD; submit as a proposal for human approval. • Route irreversible actions (promote-to-live, delete, domain change, key rotation) through WS4.
INTERFACES	in: agent tool calls · out: control-plane commands via the ERP module (same path as humans)
DATA OWNED	None — actions are logged in the shared audit_log with actor = agent + approver
SECURITY	PRD/content treated as **untrusted** (prompt-injection): the server enforces authz, not the model. No privileged backdoor.
OPERATIONS	Scoped/rate-limited AI actions; every AI write is attributable and reversible

C-08 Frontend Delivery & Block Library

ZONE B CONSUMERS

PURPOSE	The client-facing sites and the shared, typed component library they compose from.
TECHNOLOGY	Astro / Node (TS) consuming the generated TS SDK; WordPress consuming the PHP SDK. Shared block-renderer library: one FE component per backend block type (1:1).
RESPONSIBILITIES	• Render any block composition returned by the content API via the shared library. • Bespoke per-PRD design layered over the shared blocks; new blocks added once, then reused. • Rebuild static sites on publish webhook; warm CDN.
INTERFACES	in: content API (cached), form POST · out: rendered site behind CDN
DATA OWNED	None — pure consumer
SECURITY	Public read keys only; form POSTs via Turnstile; no privileged credentials on the FE
OPERATIONS	Block-library version is a contract — changes governed (§14 open item)

§06 Data Model & Multi-Tenancy

The core entities, how tenancy is enforced, and the isolation guarantees.

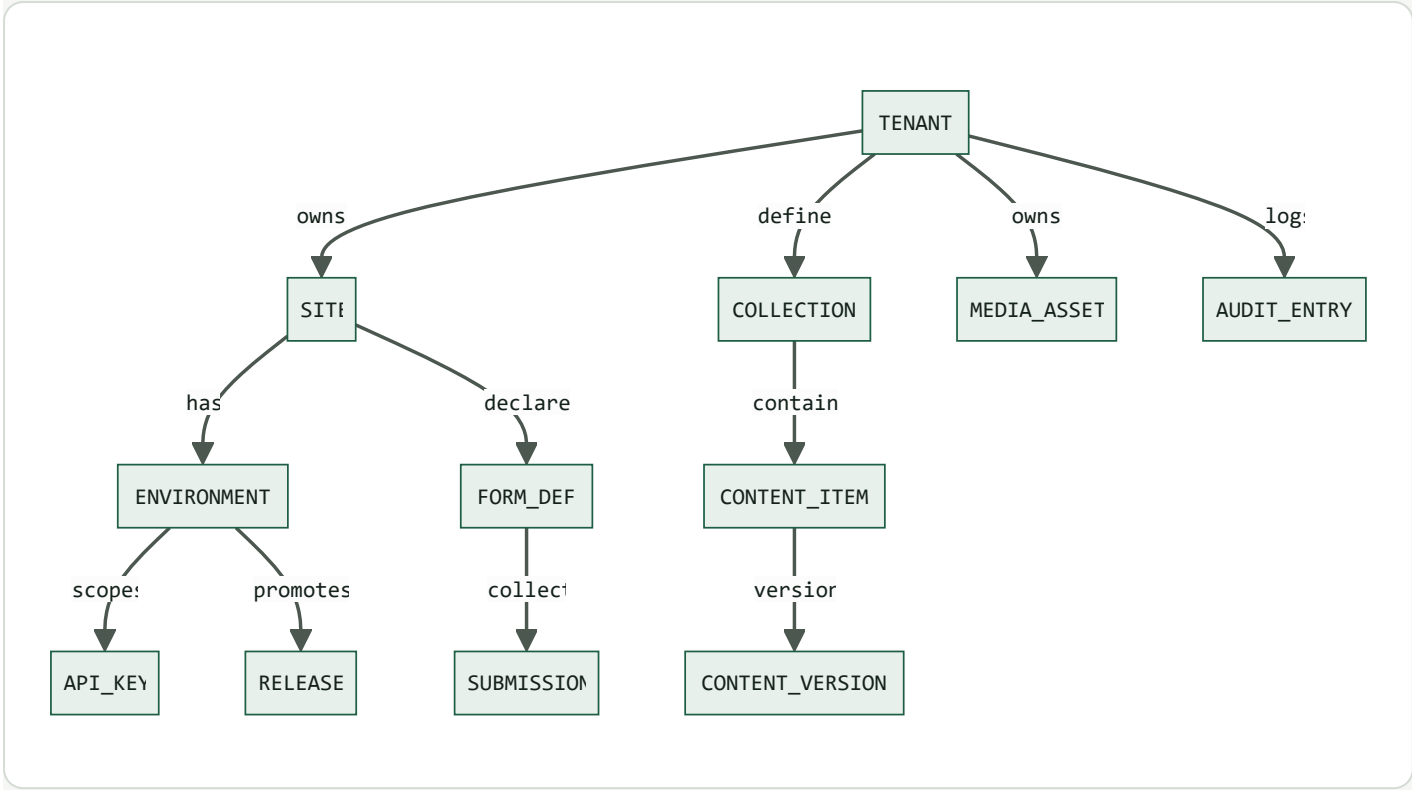

FIG. 06-1 — Core entity relationships (each arrow = one-to-many). Fields below. Every row carries `tenant_id`; RLS scopes every query.

ENTITY	KEY COLUMNS	SCOPE
TENANT	id (PK) · slug · company_scope	root
SITE	id (PK) · tenant_id (FK) · kind (astro|node|wp) · domain	per tenant
ENVIRONMENT	id (PK) · site_id (FK) · name (staging|production) · status	per site
API_KEY	id (PK) · env_id (FK) · hash · scope	per env
RELEASE	id (PK) · env_id (FK) · version · snapshot_ref (jsonb)	per env
COLLECTION	id (PK) · tenant_id (FK) · name · schema (jsonb)	per tenant
CONTENT_ITEM	id (PK) · tenant_id (FK) · collection · blocks (jsonb) · publish_state	per tenant
CONTENT_VERSION	id (PK) · content_id (FK) · version · snapshot (jsonb)	per item
MEDIA_ASSET	id (PK) · tenant_id (FK) · bucket_key · mime · size	per tenant
FORM_DEF	id (PK) · site_id (FK) · schema (jsonb) · notify	per site
SUBMISSION	id (PK) · form_id (FK) · payload (jsonb) · status	per form
AUDIT_ENTRY	id (PK) · tenant_id (FK) · actor · action · at	per tenant

Tenancy model

- **Single shared instance, tenant-scoped.** Not instance-per-client — one platform, isolation enforced in data.
- **Postgres RLS** on every table keyed by `tenant_id` (reuses the ERP's existing RLS pattern) — a query can only ever see its tenant's rows.
- **Per-tenant API keys** — hashed at rest, scoped (read/write/env), rotatable, revocable; shown once at mint time.
- **Per-tenant storage** — bucket or key prefix per tenant with scoped MinIO credentials.
- **Cerbos policies** for command-level authorization, layered above RLS.

DEFENSE IN DEPTH

Isolation is enforced at three layers — application (scoped keys), database (RLS), and authorization (Cerbos). A bug in one layer is caught by the next.

§07 Content Model & Vocabulary

How per-client flexibility and a universal contract coexist — the central design idea.

Uniformity lives in a **fixed vocabulary**; flexibility lives in **composition**; determinism comes from **codegen**. These three layers resolve the tension between every client needing something different and every frontend needing a stable contract.

LAYER 1 · VOCABULARY (FIXED)

Field primitives (text, richtext, media, relation, number, date, select, geo) · a curated set of block types (hero, richText, gallery, cta, featureGrid, form, testimonial, faq, logoCloud) · one response envelope · identical query patterns. Never per-client.

LAYER 2 · COMPOSITION (PER-CLIENT)

A client's content model is a composition of Layer-1 primitives, stored as schema data. Client A's "Case Study" and Client B's "Menu Item" are different arrangements of the same vocabulary.

LAYER 3 · CODEGEN (DETERMINISTIC)

Each tenant's schema compiles to a typed SDK, an OpenAPI doc, and a `CONTENT-CONTRACT.md` — the artifacts the FE (and Claude) build against.

The response envelope (universal)

Every content read, for every tenant, returns the same shape — so the FE never re-learns how to talk to the backend:

```
{
  "collection": "case-study",
  "slug": "acme-rebrand",
  "seo":    { "title": "…", "description": "…", "ogImage": "…"
  "meta":   { "publishedAt": "…", "updatedAt": "…" },
  "blocks": [
    { "type": "hero",        "props": { … } },
    { "type": "richText",    "props": { … } },
    { "type": "gallery",     "props": { … } }
  ]
}
```

Composition-as-data vs. schema change

SCENARIO	MECHANISM	DEPLOY?
New page using existing blocks	Compose blocks in a fixed `Page` / `Post` collection	`none — data only`
New arrangement / layout	Different block composition	`none — data only`
New structured, queryable collection	AI drafts collection config → approved diff → migrate	`approved deploy`
New block type in the vocabulary	Build once → governed contract change → reusable everywhere	`approved deploy`

The large majority of per-PRD variation is **composition-as-data** — zero deploy. Genuinely new structured collections are the minority and go through an approved, versioned deploy, which is exactly what keeps the contract deterministic.

§08 Contract & Codegen Pipeline

From a client's PRD to a typed frontend, deterministically — the pipeline that makes AI-built FEs safe.

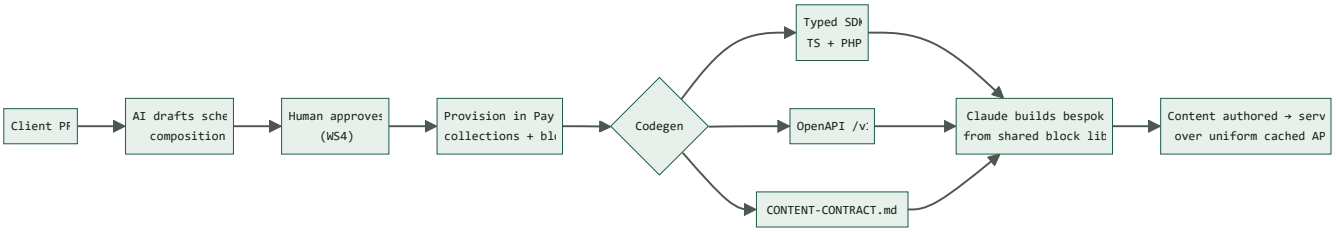

FIG. 08-1 — The contract pipeline. The FE is never built against guesses — only against compiled, typed artifacts.

Generated artifacts

ARTIFACT	CONSUMER	GUARANTEES
`TS SDK`	Astro / Node frontends	Compile-time typing of every collection, field, block
`PHP SDK`	WordPress theme	Same contract, PHP idioms
`openapi.v1.json`	Tooling, tests, external	Versioned, contract-tested surface
`CONTENT-CONTRACT.md`	Claude / FE engineers	Human-readable enumeration of the tenant's model

> **WHY THIS MATTERS**
>
> Because the FE is handed compiled types and an explicit contract, an AI (or a junior FE dev) building the frontend is working from a deterministic source, not inference. This is what makes "Claude builds the FE" reliable rather than risky.

§09 Key Flows

The four operational sequences that define day-to-day use.

9.1 · Provision a new client site (AI-drafted, human-approved)

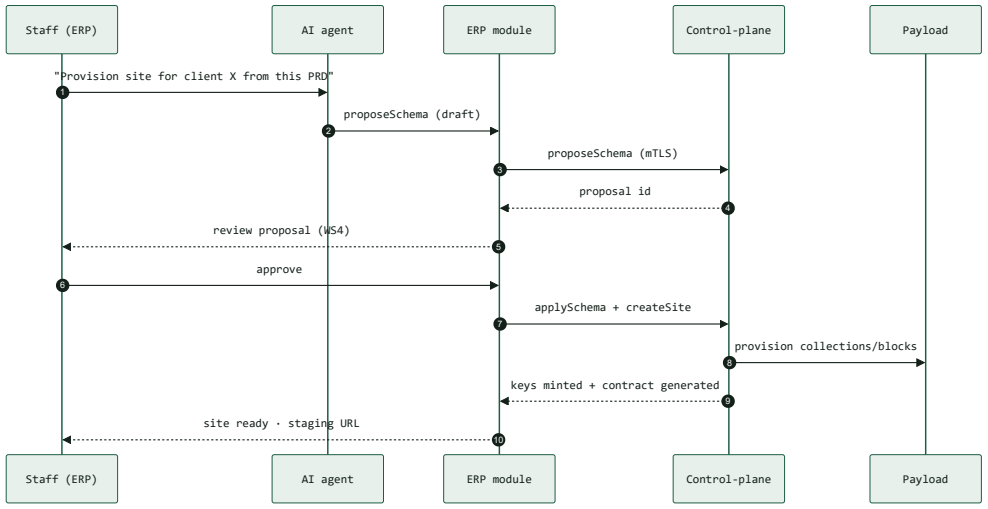

FIG. 09–1 — Provisioning. AI drafts; human approves; control-plane executes. Same path whether initiated by click or agent.

9.2 · Public form submission

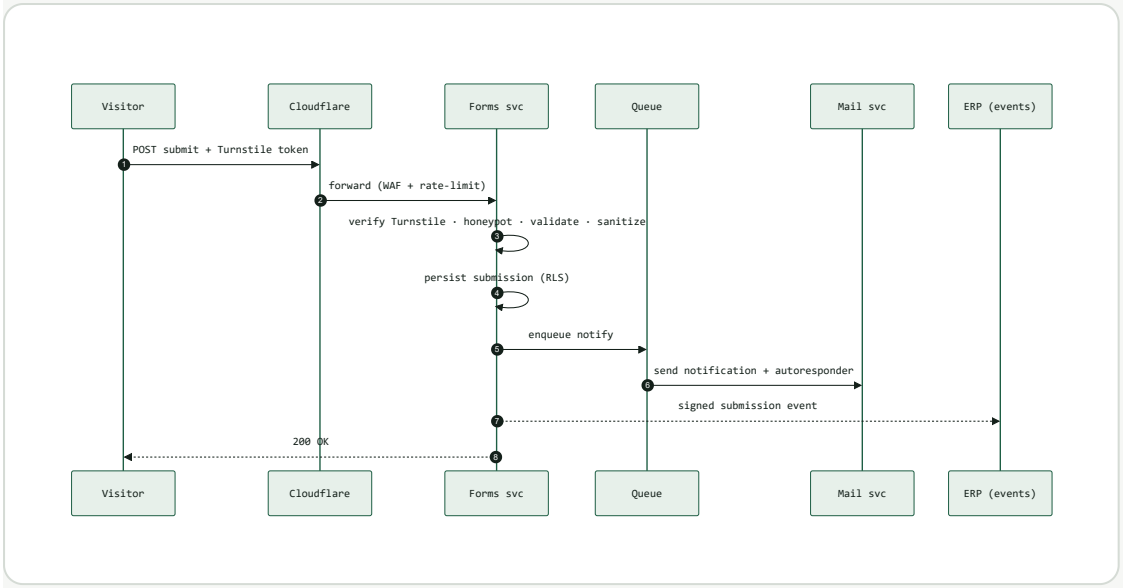

FIG. 09–2 — Form submission with layered anti-abuse before anything is persisted or sent.

9.3 · Staging → live promotion (one click, reversible)

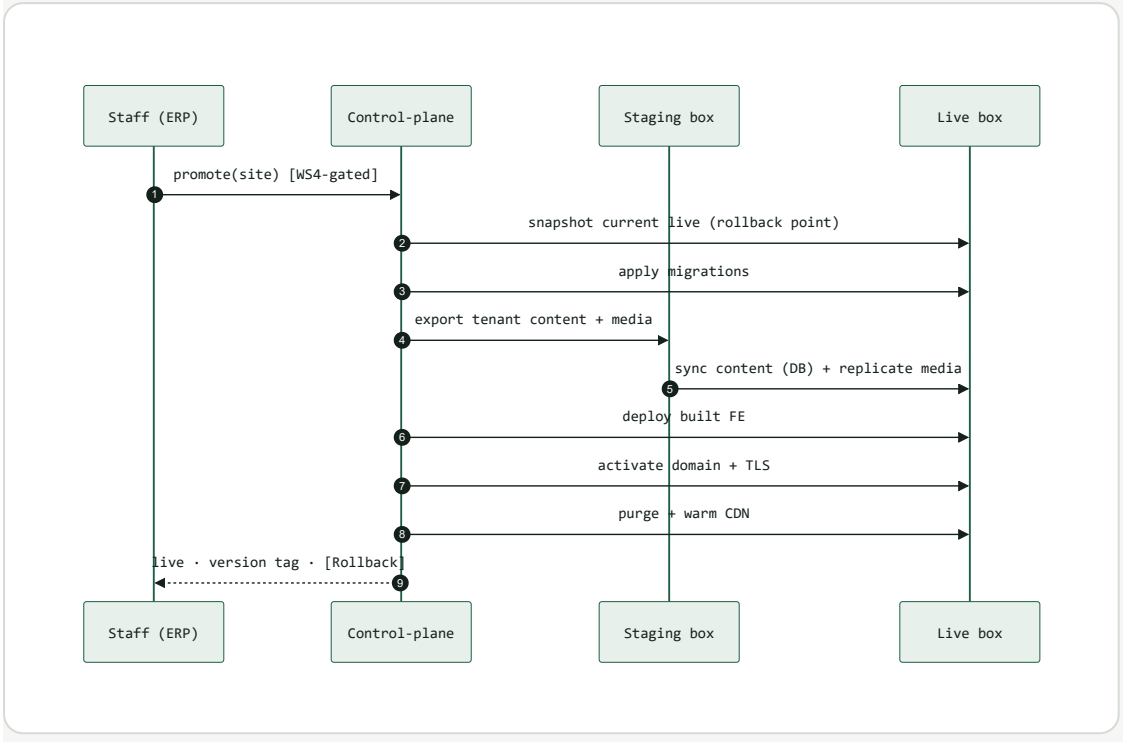

FIG. 09–3 — Promotion is a versioned release; the live snapshot taken first is the one-click rollback point.

9.4 · Content publish → static rebuild

An editor publishes in Payload → a publish webhook fires → the affected site's rebuild job is enqueued → the static build runs and deploys → the CDN cache for that tenant's tags is purged and warmed. Node/SSR sites skip the rebuild and rely on cache-tag invalidation only.

§10 Security Architecture

The threat model and the control matrix — the part that carries the most weight given the ERP connection.

Threat model

THREAT	VECTOR	PRIMARY CONTROL	SEVERITY
Pivot to company data	Compromised client site / Zone B	Two hard trust zones; no Zone B→A credentials or route	Critical
Cross-tenant data access	Broken isolation	RLS + scoped keys + Cerbos (defense in depth)	Critical
Form spam / abuse	Public POST endpoint	Turnstile + honeypot + rate limit + origin allowlist	High
Malware upload	Media upload	Auth-only + type allowlist + ClamAV + cookieless serving	High
AI performs harmful action	Prompt injection via PRD/content	Server-side authz + WS4 gate + untrusted-input stance	Critical
Credential theft	Control channel	Keycloak svc-token + mTLS; rotate; least privilege	High
DDoS	Public surface	Cloudflare edge; datastores never exposed	Medium
Supply-chain compromise	Dependencies / images	Reuse WS10: SBOM, cosign, Dependabot, image scan	High
Data loss	Single-disk MinIO / DB	Encrypted external backups; tested restore	High

Control layers

EDGE

Cloudflare WAF, DDoS, bot/rate-limit, TLS. Only `:443` public; datastores internal-only.

TENANT

RLS + scoped/hashed/rotatable keys + per-tenant storage + Cerbos policies.

INPUT

Zod validation on every API input; sanitize; size caps; strict CORS.

MEDIA

Auth-only upload, type allowlist, ClamAV, cookieless serving domain.

AI & AUTOMATION

Same authz as humans; WS4 gate on irreversible actions; PRD/content untrusted; full audit.

SECRETS

None in git; restricted-perm env now, Vault/Infisical later; keys shown once.

SUPPLY CHAIN

WS10 pipeline reuse: SBOM, cosign signing, Dependabot, container scanning.

DATA

Encryption at rest + TLS in transit; encrypted, restore-tested backups; PII retention policy.

AI SECURITY — THE NON-OBVIOUS ONE

The AI ingests client PRDs and site content, which are **untrusted input**. Defense against prompt injection is *not* in the prompt — it is that the **control-plane API enforces authorization on every command regardless of what the model decided**, and every irreversible action requires a human WS4 approval. The model can propose; only an authorized, gated command executes.

§11 Environments & Promotion

Staging and live as first-class, on separate boxes, with one-click promote and rollback.

Every site has two environments — `staging` and `production` — each with its own domain, data scope, storage prefix, and keys, deployed to **separate physical boxes** so staging load or breakage can never affect live.

Promotion contract

- **One command, gated.** `promote(site)` is a single WS4-approved control-plane command.
- **Snapshot first.** The current live state is snapshotted before any change — this is the rollback point.
- **Ordered steps.** Migrate → sync content → replicate media → deploy FE → activate domain/TLS → purge/warm CDN (see FIG. 09-3).
- **Versioned + reversible.** Each promotion produces a version tag; `rollback(site)` restores the prior snapshot in one click.
- **Preview before live.** Clients review on the staging URL first.

ASPECT	STAGING	PRODUCTION
Box	`Staging KVM8`	`Live KVM8`
Purpose	Author, preview, client review	Public traffic
Domain	`staging.<client>`	`<client> production`
Promotion	source	target (WS4-gated)
Rollback	—	one-click to prior snapshot

> **OPEN ITEM – CONTENT SYNC MECHANISM**
>
> The exact staging→live content transfer (Payload export/import vs. tenant-scoped DB sync vs. publish-target flags) is flagged in §14 for a focused design decision before Phase 4.

§12 Observability & Operations

How we see the platform, back it up, and keep it running — reusing what the estate already has.

Telemetry

- **Reuse WS9** — OpenTelemetry across all Zone B services, exported to the existing Grafana stack in Zone A (fail-soft via `OTEL_ENABLED`).
- **Golden signals** per service: request rate, error rate, latency, saturation.
- **Alerts**: form-submission spikes, auth-failure spikes, unusual egress, queue depth, delivery failures, backup failures.

Backups & DR

ASSET	METHOD	TARGET	CADENCE
Postgres (Zone B)	Logical dump + WAL	External (R2/B2), encrypted	Daily + continuous
MinIO objects	Bucket replication	External S3/R2	Continuous
Config / secrets	Sealed / vault export	Secure store	On change
Restore drill	Rehearsed restore to staging	—	Quarterly

> **SINGLE-DISK CAVEAT**
>
> MinIO on a single VPS disk has no built-in redundancy — external replication is not optional; it is the redundancy. Treat the VPS copy as primary and the external replica as the recoverable backup.

CI/CD & release

Builds run through the **WS10 supply-chain pipeline** (SBOM, cosign, image scan). The ERP *promote/deploy/rebuild* buttons trigger release jobs; every release is versioned and attributable.

§13 Roadmap, Phases & Milestones

Dependency-ordered phases, each shippable, each with a clear gate before the next.

P1 **Foundation**

Rebranded Payload + dedicated Postgres with RLS + API-key auth + uniform response envelope + MinIO storage. Proof: one test tenant, one collection, one Astro site reading real content.

`GATE → a real page renders from the central API through a scoped key. · Zone B skeleton stood up.`

P2 **Forms & Mail — the web3forms kill**

Forms service (Turnstile, validation, persistence) + Mail service (provider adapter, queue, retries). Highest immediate value, smallest surface.

`GATE → a live client form submits centrally, notifies, and autoresponds. · First production dependency migrated off web3forms.`

P3 **Contract & codegen + block library v0**

OpenAPI, typed TS SDK, `CONTENT-CONTRACT.md` generator, shared block-renderer library first set.

`GATE → a new site is built purely from generated types + shared blocks, no hand-written backend calls.`

P4 **Control plane in the ERP + environments**

WebDev console: site registry, provisioning, key management, staging/live promotion + rollback. Control-plane API + mTLS/Keycloak seam. (Resolve content-sync open item first.)

`GATE → provision, promote, and roll back a site entirely from ERP clicks.`

P5 **AI execution + approvals**

MCP tools over the command set, AI schema-drafting from PRD, WS4 approval gating, full audit.

`GATE → an agent provisions a site from a PRD, human-approved, fully audited.`

P6 **WordPress full-headless**

PHP SDK + headless theme pattern. Last, because heaviest — proven model first on Astro/Node.

`GATE → a WordPress site renders entirely from the central content API.`

Indicative sequencing

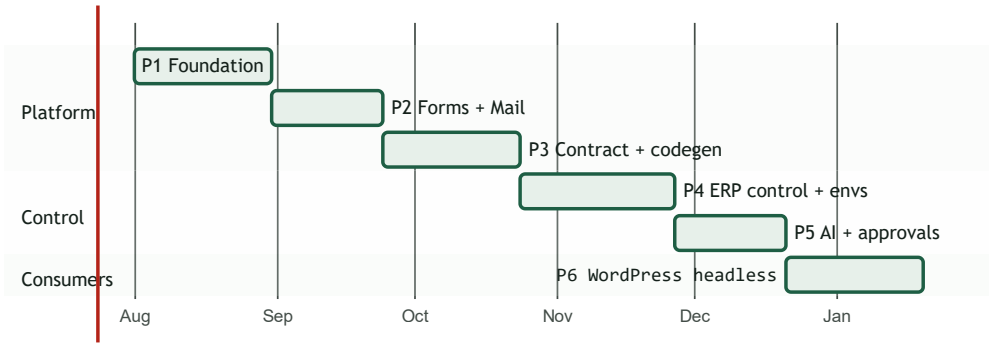

FIG. 13-1 — Indicative timeline; durations are planning estimates, not commitments. Sequencing (dependencies) is the fixed part.

Sample ticket decomposition (Phase 1)

ID	TICKET	TIER	DEPENDS ON
P1-01	Zone B compose stack skeleton (proxy, payload, api, pg, minio, redis)	`medior`	—
P1-02	Payload rebrand + Postgres adapter + tenancy fields	`senior-be`	P1-01
P1-03	RLS policies + roles/grants for content tables	`senior-db`	P1-02
P1-04	API-key mint/verify + scoped read middleware	`senior-be`	P1-02
P1-05	Uniform response envelope + first block types	`medior`	P1-02
P1-06	MinIO storage adapter + media upload path	`medior`	P1-01
P1-07	Test Astro site reading real content (proof)	`junior`	P1-04, P1-05
P1-08	QA: tenancy isolation + envelope contract tests	`qa`	all

Full per-phase decomposition (with Opus-tagged tickets per the agent-army standard) is produced by the architect at plan time and fed to `/army`.

§14 Risks, Open Items & Decision Log

Open items (to resolve before their phase)

ITEM	DECISION NEEDED	NEEDED BY
Staging→live content sync	Payload export/import vs. tenant DB sync vs. publish-target flags	P4
Headless preview/draft	Preview mechanism for bespoke FEs (Payload draft mode + FE preview routes)	P3
Block-library governance	Who approves a new block into the vocabulary (contract change process)	P3
Cache invalidation	Per-tenant cache-tag scheme + publish-driven purge granularity	P3
Custom domains + TLS	Per-tenant domain onboarding + automated cert issuance on KVM8	P4

Risks & mitigations

RISK	IMPACT	MITIGATION
Single VPS bandwidth for media/video	Slow sites, cost	CDN hard-cache reads; route video to R2
MinIO single-disk data loss	Media loss	Mandatory external replication + restore drills
Schema-change friction (deploy per new collection)	Slower onboarding	Maximize composition-as-data; batch schema changes
WordPress full-headless effort	Timeline	Sequenced last; model proven on Astro/Node first
Contract drift across sites	FE breakage	Versioned `/v1`; contract tests in CI

Decision log (locked)

#	DECISION	RATIONALE
D1	All sites full-headless (incl. WordPress)	Uniform consumption model; WP last (heaviest)
D2	Payload 3 as headless engine, rebranded	MIT (no revenue clause), code-first schema fits codegen, native blocks, local API
D3	Vocabulary + composition + codegen content model	Resolves per-PRD flexibility vs. uniform contract
D4	Bespoke FE from shared typed block library	Design freedom + deterministic, fast connection
D5	AI drafts schema, human approves (WS4)	Autonomous-leaning but safe
D6	Shared instance + RLS + per-tenant keys	Scales on one box; reuses ERP pattern
D7	TypeScript end-to-end (PHP only for WP SDK)	One language, shared types, staff mobility
D8	MinIO (VPS disk) + R2 (video) + Cloudflare CDN	Uses Hostinger storage; S3-portable; protects bandwidth
D9	Two hard trust zones	Contain breach; internet platform never a path to company data
D10	3 boxes: ERP + staging + live	Physical isolation + true staging/live separation
D11	Keycloak client-credentials + mTLS control channel	Reuses infra; strongest practical service auth
D12	One-way control, signed-webhook events	No cross-zone DB grants
D13	Versioned, reversible one-click promotion	Safe staging→live for humans and AI alike

§15 Glossary

TERM	MEANING
Zone A / Zone B	Internal ERP trust zone / internet-facing website-platform trust zone
Vocabulary	The fixed set of field primitives and block types shared by all tenants
Composition	A tenant's content model as an arrangement of vocabulary — stored as data
Envelope	The universal response shape every content read returns
Control-plane	The single command API for all lifecycle operations
WS4 approvals	Existing ERP human-approval surface reused to gate irreversible actions
WS9 / WS10	Existing observability stack / supply-chain-secure release pipeline, reused
Turnstile	Cloudflare's privacy-first CAPTCHA, used on public form posts
RLS	Postgres Row-Level Security — per-tenant query scoping

GAIADA WebDesk — Engineering Blueprint v1.0

Status: Draft for review · 2026-07-23 · Web Dev · Platform ·
Classification: Internal
Architecture decisions locked (§14). Feeds architect decomposition → /army. Not a commitment of dates; sequencing is fixed, durations are estimates.